BTP Proposal: A Multi-Agent Generative AI System for Time-Series Forecasting and Anomaly Detection Across Astronomy and Financial Domains

Student: Kaveri Neeli **Duration:** 3 months (2026-07-26 to 2026-10-25) **Mode:** Solo BTP

1. Objective

Develop a multi-agent AI system that performs time-series analysis across two structurally different domains, sharing a common agent-orchestration and retrieval-augmented reasoning layer:

- **Astronomy:** anomaly detection in light curve data (public Kepler/TESS datasets)
- **Financial markets:** stock price prediction using historical data (Yahoo Finance) combined with sentiment analysis from news and social sources

The system is composed of specialized agents (data-retrieval, domain-modeling, sentiment, and explanation agents) coordinated by an orchestrator, with a shared vector-embedding retrieval layer connecting both domains.

2. Novelty / Why Two Domains

Light curves and financial time series differ in noise structure, periodicity, and "anomaly" semantics (a transit dip vs. a sentiment-driven price shock), which makes a shared multi-agent architecture a genuine test of generality rather than a single-domain pipeline. The project also contributes a **self-trained time-series embedding** for light curves (no standard pretrained equivalent exists, unlike text), alongside free pretrained text embeddings for the financial-sentiment side.

3. Development Plan (free / open-source, no cost during build phase)

All development work will use free, open-source, or public-domain resources — no paid subscriptions are needed until the final demonstration:

Component	Free resource used
LLM reasoning (agents)	Open-source models via **Ollama** (e.g. Llama 3.2, Qwen2.5) run locally
Text/document embeddings	Open-source pretrained models (e.g. Sentence-Transformers MiniLM) run locally — no paid embedding API
Time-series embeddings	A small encoder trained by me on light-curve data — a project deliverable in its own right
Astronomy data	Public NASA Kepler/TESS light curves (via the open-source `lightkurve` library)
Financial data	Yahoo Finance historical data (`yfinance`)
News/social sentiment	Public RSS feeds and free-tier Reddit API — no paid news API
Vector storage / retrieval	Local, open-source vector store (FAISS/Chroma)
Agent orchestration	Open-source framework (LangGraph)

Known trade-off (as flagged in my original email): open-source LLMs run locally are free but slower than commercial APIs, especially on a CPU-only laptop. To address this rigorously rather than just asserting it, I will run a **controlled benchmark comparing my local laptop setup against a lab parallel-computing system** (see §5) — same models, same workload, measured latency and throughput. This benchmark is itself one of the project's deliverables, directly answering the open question in my proposal about which setup is most suitable.

4. Demonstration Plan (final presentation only)

For the live demo, I will temporarily switch the system's LLM backend from local Ollama models to a cloud API (OpenAI or Claude) to guarantee fast, reliable responses in front of an audience. This is a one-line configuration change in the system (the architecture is built to be model-agnostic from the start), not a different system — so the demo faithfully represents what was built, just with a faster "engine" swapped in for the day.

This is the only point in the project where a paid service is used, and only briefly (final testing + the demo itself).

Resource request

I would need access to an OpenAI or Claude account for this final-demo phase (Month 3, roughly Weeks 11–13, i.e. early-to-mid October 2026 onward). I'm asking for guidance on the best way to arrange this — whether through a departmental account, credits, or another supported route.

5. Compute Request

I will begin and carry out most development on my personal laptop (CPU only), which is sufficient for the small open-source models and classical ML components this plan uses. As discussed, I'd like to request access to the lab's parallel-computing system around **Month 3, Week 10 (week of Sep 28, 2026)**, specifically to:

1. Run a head-to-head timing benchmark (laptop CPU vs lab system) on identical workloads, and
2. Have a faster fallback available if any component (e.g. the light-curve autoencoder training) proves too slow locally.

I don't anticipate needing lab access for the full 3 months — mainly for this benchmarking and contingency window.

6. Milestone Timeline (for progress check-ins)

Milestone	Target date	What I'll show
M1: Domain MVPs	~Aug 23, 2026	Standalone astronomy anomaly detector + standalone financial predictor with sentiment, each with evaluation metrics
M2: Multi-agent system integrated	~Sep 27, 2026	Full multi-agent pipeline running locally end-to-end on the open-source stack, for both domains, through a simple front-end
M3: Benchmarked + final demo-ready	~Oct 25, 2026	Local-vs-lab performance comparison, evaluation results, cloud-API demo path tested, final report and slides

I'd propose brief check-ins around each milestone date so I can course-correct early if something isn't working.

7. Expected Deliverables (end of 3 months)

1. A working multi-agent system covering both astronomy anomaly detection and financial forecasting with sentiment
2. A self-trained light-curve embedding model
3. Evaluation results: anomaly-detection precision/recall/AUC (astronomy), RMSE with vs. without sentiment features (finance)
4. A quantitative comparison of local (CPU) vs. lab (parallel-computing) performance for the same workloads
5. Final report and a live demo (cloud-API-backed for responsiveness)

8. Summary of What I'm Asking For

- **Now:** nothing — development proceeds on free/open-source tools on my own laptop.
- **~Week 10 (late Sep 2026):** access to the lab parallel-computing system for benchmarking and as a training fallback.
- **~Week 11 onward (early-mid Oct 2026):** an OpenAI or Claude account, used only for final testing and the live demonstration.

I'd appreciate your feedback on this plan, particularly on the proposed milestone dates and the best way to arrange lab/API access when the time comes.

Sincerely, Kaveri Neeli